

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

MEDICONNECT

Hospital Management & Appointment Booking System

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)

Academic Year: 2026–2027

Team Member	Roll Number
Tejasreddy K	2520080027
Guru Charan S	2520090061
Lokesh T	2520090105

Guide: _____

Section / Team: _____

KLH Bachupally Campus, CSE

CHAPTER 0: DECLARATION

Declaration

The project report entitled “**MediConnect: Hospital Management & Appointment Booking System**” is a record of bonafide work carried out by Tejasreddy K (2520080027), Guru Charan S (2520090061), and Lokesh T (2520090105) in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), during 2026–2027 at KLH University.

The report describes the project architecture, implementation, database integration, API testing, deployment procedure, challenges, and future enhancements.

CHAPTER 0: ACKNOWLEDGEMENT

Acknowledgement

We express our sincere gratitude to the faculty members, course coordinator, project guide, and the Department of Computer Science and Information Technology for their guidance and support throughout the development of MediConnect.

We also thank the teaching and non-teaching staff, classmates, friends, and family members whose support helped us complete the project and its documentation.

CHAPTER 0: ABSTRACT

Abstract

MediConnect is a web-based Hospital Management and Appointment Booking System designed to centralize common hospital workflows for patients, doctors, and administrators. The system provides a structured platform for patient registration, doctor management, appointment booking, and hospital-related information.

The frontend uses HTML, CSS, and JavaScript. The backend is implemented using Java 21 and Spring Boot with REST APIs, while MongoDB provides persistent storage. Apache Maven manages the backend build and Postman is used to test APIs.

The working backend was verified on port 8080. The health endpoint returned an UP status and the doctor-management API successfully created a doctor record and returned JSON data. The system can be extended with medical records, prescriptions, billing, departments, notifications, and stronger role-based security.

INDEX

S.No.	Chapter	Page
1	Introduction	5
2	System Requirements	7
3	Technology Stack	8
4	System Architecture	9
5	Design and Database	10
6	Implementation	12
7	Features	14
8	Testing	16
9	Deployment	18
10	Challenges & Limitations	19
11	Future Enhancements	20
12	Conclusion	21
13	References	22
14	Appendices	23

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Hospitals manage patient registration, doctor information, appointments, medical records, prescriptions, billing, and departments. When these activities are handled through disconnected or manual processes, information can be difficult to track and update.

MediConnect centralizes the major workflows in one web application. A browser frontend communicates with a Spring Boot REST backend, which persists data in MongoDB.

1.2 Problem Statement

The project addresses the difficulty of coordinating patient and doctor information and appointment operations through fragmented workflows. A centralized system is required to make these services easier to access and manage.

1.3 Scope

The scope covers a browser-based frontend, Spring Boot REST services, MongoDB persistence, doctor management, patient-oriented workflows, and appointment booking. The architecture is extensible toward medical records, prescriptions, billing, and department management.

1.4 Objectives

- Centralize hospital-management operations.
- Provide patient registration and login workflows.
- Provide doctor information and management.
- Enable appointment booking.
- Store application data in MongoDB.
- Provide testable REST APIs.

1.5 Importance

The application reduces fragmented manual workflows and provides a common interface for hospital-related operations.

1.6 Target Users

Patients/users, doctors, and administrators are the primary target users.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

A modern laptop or desktop with at least 4 GB RAM and adequate disk space is sufficient for the prototype. The frontend, Spring Boot backend, and MongoDB can be run locally for demonstration.

2.2 Software Requirements

• Java 21 JDK • Apache Maven 3.9.x • MongoDB • Chrome/Edge/Firefox • Visual Studio Code • Postman • Windows 10/11 or another supported OS

2.3 Development Tools

Spring Boot, Maven, MongoDB, HTML/CSS/JavaScript, VS Code, and Postman are the main development and testing tools.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

HTML, CSS and JavaScript provide the browser-based interface, including login, registration, dashboard, doctors, and appointment-oriented pages.

3.2 Backend

Java 21 with Spring Boot provides the REST API and embedded Tomcat server. Apache Maven manages dependencies and build execution.

3.3 Database

MongoDB provides document-oriented persistent storage. The running application successfully connected to MongoDB through the MongoDB Java driver and Spring Boot integration.

3.4 Development and Version Control

VS Code is used as the development environment. Git/GitHub can be used to maintain and submit the source code.

3.5 API Testing

Postman is used to send JSON requests and inspect HTTP responses. Browser testing was also used for the health endpoint.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The system follows a three-layer flow: Frontend → Spring Boot REST API → MongoDB. The frontend never needs direct access to the database; requests are handled through backend APIs.

4.2 Presentation Layer

HTML/CSS/JavaScript pages collect user input and display responses received from the backend.

4.3 Application Layer

Spring Boot REST controllers receive JSON requests, validate/process them, and return JSON responses.

4.4 Data Layer

MongoDB stores persistent application documents such as doctor and appointment information.

4.5 Runtime Architecture

For local development the Spring Boot backend runs on port 8080 and MongoDB commonly runs on port 27017. The browser communicates with the backend through HTTP REST requests.

CHAPTER 5: DESIGN AND DATABASE

5.1 Database Design Principles

Major resources are kept as separate document structures so that the application can evolve without requiring a rigid relational schema. Identifiers are used to associate related records where required.

5.2 Main Data Entities

The project includes users/patients, doctors, appointments, and extensible hospital information. Future modules can represent medical records, prescriptions, billing, and departments.

5.3 Design Decisions

MongoDB was selected for flexible document storage. REST APIs provide a clear boundary between the frontend and backend.

5.4 Verified API Resource

The tested POST `/api/doctors` endpoint accepts fields including name, department, specialization, rating, experience, and availability and returns an identifier with the stored doctor data.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The backend is a Spring Boot application with REST resources for health checking and hospital operations. Frontend pages provide user-facing workflows for registration, login, doctors, dashboard, and appointments.

6.2 Frontend Logic

JavaScript collects form data, sends HTTP requests, and renders JSON responses. HTML and CSS provide the page structure and presentation.

6.3 API Integration

Frontend actions communicate with the backend using JSON over HTTP. Postman was used to test backend resources independently.

6.4 Authentication and Authorization

Login and registration interfaces are included in the application. Role-based authorization can be expanded using Spring Security so that patient, doctor, and administrator operations are separated.

CHAPTER 7: FEATURES

7.1 Main Features

- Patient/user registration and login
- Doctor listing/management
- Doctor details such as department, specialization, rating, experience and availability
- Appointment booking
- Dashboard-oriented interface
- REST APIs
- MongoDB persistence
- Health monitoring via /api/health

7.2 Feature Workflow

A user enters information in the frontend. JavaScript sends the information as JSON to the corresponding Spring Boot endpoint. The backend processes the request, interacts with MongoDB, and returns a JSON response.

During testing, POST /api/doctors returned HTTP 200 and a created doctor object with an assigned id. GET /api/health returned the service status as UP.

CHAPTER 8: TESTING

8.1 Postman API Testing

Postman was used to test REST endpoints independently of the frontend. The doctor creation endpoint was tested with a JSON request body.

8.2 Integration Testing

The backend was tested while Spring Boot, embedded Tomcat, and MongoDB were running together. Startup logs confirmed successful database connectivity and web-server startup.

8.3 Tools Used

• Postman • Browser • Maven • MongoDB • VS Code

8.4 Test Cases and Results

1. GET /api/health — expected UP — verified successfully. 2. POST /api/doctors with valid JSON — expected created doctor JSON — verified with HTTP 200. 3. Spring Boot startup — expected Tomcat on port 8080 — verified in terminal logs. 4. MongoDB connection — expected successful connection — verified in application logs.

CHAPTER 9: DEPLOYMENT

9.1 Local Deployment Steps

1. Install Java 21 and Maven. 2. Start MongoDB. 3. Open the backend directory containing pom.xml. 4. Run mvn spring-boot:run. 5. Wait for Tomcat to start on port 8080. 6. Open the frontend. 7. Verify <http://localhost:8080/api/health>.

9.2 Environment Configuration

Database and application configuration should be stored in application.properties or environment-specific settings. Secrets should not be committed to public repositories.

9.3 Hosting

The current implementation is suitable for local development and demonstration. A future production deployment can host the frontend separately and deploy the Spring Boot backend with managed MongoDB.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced

Java/Maven configuration, correct project-directory selection, Spring Boot plugin resolution, and MongoDB connectivity required troubleshooting during development.

10.2 Solutions Applied

Java was verified with `java -version`, Maven with `mvn -version`, Maven was configured in the environment, and the backend was run from the directory containing `pom.xml`. MongoDB connectivity was confirmed in Spring Boot logs.

10.3 Current Limitations

The current project is primarily a local prototype. Production-grade security, complete medical records, billing, notifications, comprehensive role management, and cloud deployment can be expanded.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

• Medical-record management • Prescription management • Billing • Department management • Role-based access • Email/SMS notifications • Appointment reminders

11.2 Integrations and Optimizations

Cloud deployment, automated tests, payment integration, calendar integration, analytics dashboards, stronger audit logging, and improved validation can be added.

CHAPTER 12: CONCLUSION

12.1 Summary

MediConnect provides a centralized web-based approach to hospital management and appointment booking using a frontend, Spring Boot REST backend, and MongoDB.

12.2 What Was Achieved

The project environment was successfully configured with Java 21 and Maven. Spring Boot started on port 8080, MongoDB connected successfully, the health API returned UP, and doctor creation was verified through Postman.

12.3 Skills Learned

The project provided practical experience in Spring Boot, REST API development, MongoDB integration, Maven, JSON communication, frontend-backend integration, Postman testing, and debugging.

CHAPTER 13: REFERENCES

- [Spring Boot Official Documentation](#)
- [Spring Framework Official Documentation](#)
- [MongoDB Official Documentation](#)
- [Apache Maven Official Documentation](#)
- [Postman Official Documentation](#)
- [MDN Web Docs — HTML, CSS and JavaScript reference](#)

CHAPTER 14: APPENDICES

Appendix A: Health API

GET `http://localhost:8080/api/health`

Example response: `{"status":"UP","service":"MediConnect Backend"}`

Appendix B: Doctor API

POST `http://localhost:8080/api/doctors`

`{"name":"Dr. Rahul Kumar","department":"Cardiology","specialization":"Cardiologist","rating":4.9,"experience":10,"available":true}`

Appendix C: Team Members

Tejasreddy K — 2520080027

Guru Charan S — 2520090061

Lokesh T — 2520090105

Appendix D: Final Submission Note

Add the latest frontend, Postman, MongoDB, and application screenshots before final printing/submission. Fill the guide, section, and team details on the cover page.

PROJECT SUBMISSION CHECKLIST

Item	Status
Frontend	Included in project
Spring Boot backend	Implemented
MongoDB	Connected and verified
Health API	Verified
Doctor API	Verified with Postman
Appointment module	Project scope
Final screenshots	Add latest screenshots
Guide / Section	Fill before submission